

Benchmarking Project

Computational Thinking an Algorithms

Samantha Kenny

Table of Contents

Introduction	2
What is an Algorithm?.....	2
Algorithm efficiency	2
Analysing Efficiency.....	2
Sorting Algorithms	5
What is Sorting?	5
In place sorts.....	5
Bubble Sort	6
Selection Sort	11
Insertion Sort	15
Efficient Comparison Sorting Algorithms	19
Merge Sort	19
Non-Comparison Sorting Algorithms	24
Counting Sort	24
Implementation and the Benchmarking process	29
Java Benchmarking main function code	30
Benchmarking results for the five sorting algorithms	32
References	33
Reference list	33

Introduction

In this project I will test and compare the relative performance of sorting algorithms, in order to demonstrate a full analysis of algorithms based on given datasets, and how the various types of algorithms perform based on their complexity and performance.

I will discuss the various types of sort algorithms I have used, how they work on the data in order to sort and compare the times taken for each type and the efficiency of the algorithm based not only on the time taken, but also on the complexity of the data used.

The references used for this project was predominantly the lecture notes provided for the Computational Thinking with Algorithms module at ATU and Data Structures using Java by Bradley N. Miller, David L. Ranum, Roman Yasinovskyy, J. David Eisenberg and various websites that will be listed in the references section.

What is an Algorithm?

In computing terms an algorithm is a list of instructions that a computer follows in order to perform a task.

The computer follows the list of instructions in a FIFO (First In, First Out) order and any algorithms written, should be done so with this in mind.

A good Algorithm should perform the task quickly and use the least amount of resources possible, while producing correct and accurate outputs based on the inputs.

Algorithms can vary in the both the time taken and the resources used, even if the inputs are the same and the desired output is the same, as this project will show.

Algorithm efficiency

There are two main measurements for algorithm efficiency.

- Time Efficiency

This considers the time, or the number of operations needed for the computer to run an algorithm.

- Space Efficiency

This considers the amount of memory or storage the computer needs to run the algorithm.

Analysing Efficiency

Algorithms are platform independent and can be implemented using any computing language, however results obtained can be dependent on the hardware and software used and due to being platform independent, there needs to be an independent form of comparison also.

Algorithms can be theoretically analysed using a Priori analysis and empirically analysed by a Posteriori analysis.

Posteriori analysis evaluates efficiency empirically, the relative efficiency is analysed by comparing actual measurements collected during experimentation and is a measure of performance. External factors like the computer architecture can affect performance, as can the complexity.

Priori analysis does not take into account the implementation of the analysis and the impact the computer architecture will have on the algorithm. It uses the order of growth to measure the efficiency with respect to the size of an input (n) and is a measure of complexity. The order of growth refers to how much an algorithm grows based on how big a data set is given as an input. The order of growth increases as the data set size increases as more operations will be needed in order to do the work required by the algorithm.

Complexity measures an algorithms efficiency with respect to internal factors, such as time needed to run an algorithm. The complexity can alter the performance of an algorithm. Algorithms can be compared by evaluating their running time complexity on an input data set of size n and can be used to estimate the time that will be needed as the data sets scale in size up and down in relation to the initial size n and whether the algorithm would be suitable for use as the data set grows. Typically, complexity falls into one of a number of families based on the order of growth being a certain order.

Complexity is measured through the order of magnitude using Big-O notation.

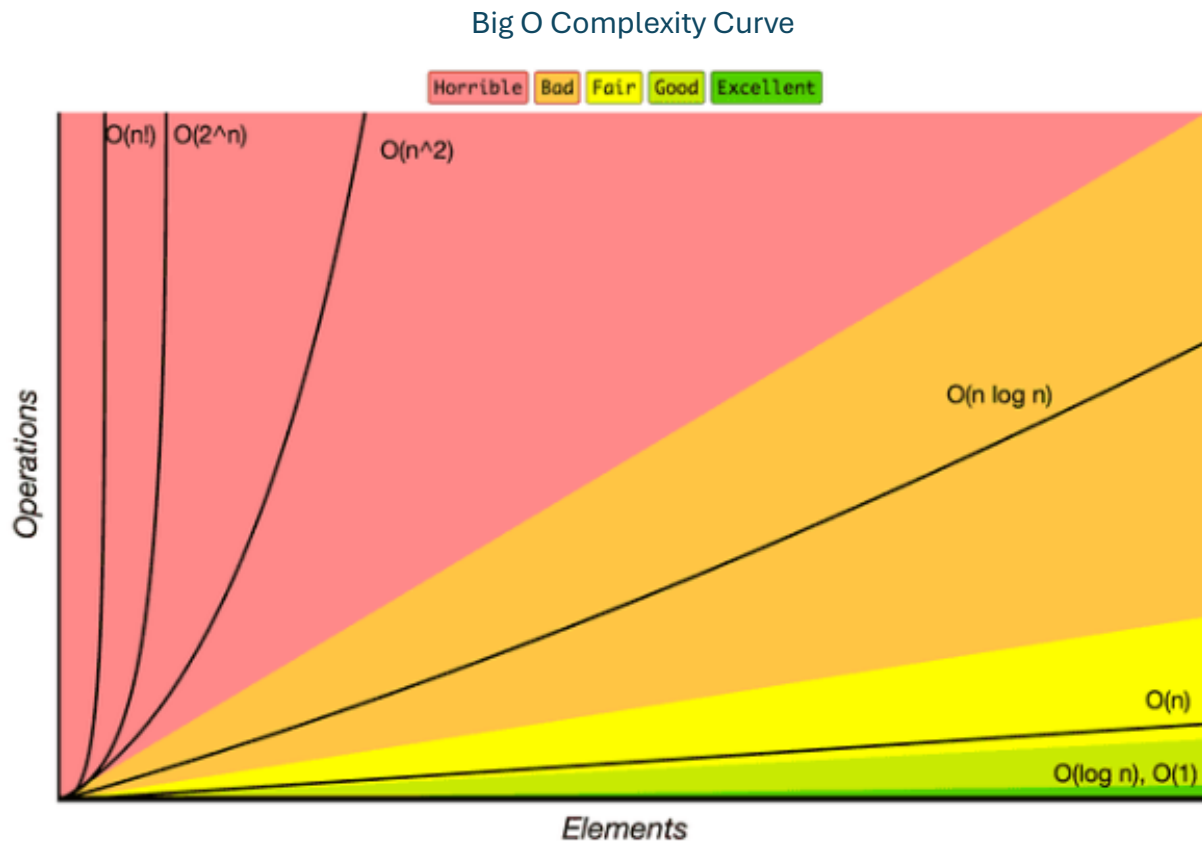
Big-O notation represents the relationship between the size of the input and the number of operations required to solve the problem.

The most common functions for Big-O are shown in the table below.

Common Functions for Big O
([DS Big O Notation \(runestone.academy\)](https://runestone.academy/))

f(n)	Name
1	Constant
Log n	Logarithmic
n	Linear
$n \log n$	Log linear
n^2	Quadratic
n^3	Cubic
2^n	Exponential

Big O notation is used to describe the complexity of an algorithm based in the worst-case scenario. It takes into account the external factors that impact the algorithmic performance and returns a complexity curve as shown below.



As shown by the graph, though initially there seems to be little differences between the various functions, as the data set grows the efficiency wildly differs and there are very clear differences demonstrating, the best, worst and average cases for algorithms.

Worst-case defines a class of input instances for which an algorithm exhibits its worst runtime behaviour. They are denoted by the O notation for example $O(n \log n)$. The worst-case scenario can be used to describe either the execution time required, or the space used while executing.

If algorithms have similar Big O notation, their complexity is similar in terms of their time or space requirements for the worst-case scenario.

If one algorithm has a less complex Big O notation than another it is inferred it is much more efficient in terms of time or space requirements for the worst-case scenario

Best-case defines a class of input instances for which an algorithm exhibits its best runtime behaviour.

Average-case defines the expected behaviour when executing the algorithm on random input instances.

On the graph the worst-case scenarios would include algorithms of the order n^2 , 2^n , and $n!$

Best case scenarios would include constant (1) and logarithmic ($\log n$). Omega (Ω) notation is used for best case complexity, for example $\Omega(n)$. They are rare in occurrence and represents a lower number of operations need for the work.

Average case scenarios would include algorithms of the order n and $n \log n$.

When an algorithm running time is no greater, or less than a certain order theta (θ) notation is used. This can be used for example, if an algorithm is $\theta(n)$, it is both $O(n)$ and $\Omega(n)$, which is inferring it refers to algorithms that fall under the average case scenario.

In this project I am testing the efficiency of these algorithms. I wrote code in Java language that created the relevant sorting algorithms, I increased the size of the array as I tested each algorithm and for each sized array, I did 10 repetitions and took the average time the algorithm took to run.

Sorting Algorithms

What is Sorting?

Sorting is the process in which specified data is arranged in a specific order based on a specified criterion. In this case I will be sorting randomised numbers, into ascending order.

Sorted data is beneficial in the computing world. Having data pre-sorted can increase efficiency of programmes and in turn make programmes more desirable for use as they may take up less resources and take less time.

Sorting Algorithms are algorithms specifically designed to sort various types of data in a specific method. Some may have memory requirements and thus will have a higher overhead in order to execute, others will not, meaning that at lower sized data sets they are extremely efficient but as the data set size grows the efficiency will dramatically decrease.

Most sorting algorithms are deemed as comparison-based algorithms, that is algorithms that work by comparing the elements within the data set and sorting them based on the results of that comparison.

The main types of sorting algorithms are.

- In place sorts
- Efficient comparison sorts
- non-comparison sorts
- Hybrid sorts

In place sorts

These sorting algorithms work by sorting an array of elements without using any additional memory.

They do not create new arrays to place the sorted elements in to, the name “in place” comes from the sorting algorithm literally doing just that and sorting the elements in place without moving elements in or out of the array. This is done by using comparison and moving the elements within the same array, based on the results of that comparison.

In this project we will use two different types of in place sorts

- Bubble sort.
- Selection Sort

Bubble Sort

Bubble sort is a sorting algorithm that works by comparing adjacent elements and based on the result of that comparison either switching them or leaving them where they are. The sorting algorithm while iterating through the array of numbers only compares the pair of numbers it is looking at before moving on to the next pair. It doesn't compare those numbers with any numbers previously parsed, and it doesn't hold on to those numbers to allow any of the numbers it is yet to parse a comparison with them.

It is classed as a simple in-place comparison-based algorithm as it does not create a new array to place the sorted elements in to, the algorithms used are simple in nature and it compares the elements two at a time within the array and based on that comparison the decision is taken to either move the element or leave it where it is.

The reason it is called a bubble sort is that the way the algorithm works is to basically bubble along the array two numbers at a time comparing them and if necessary, moving them.

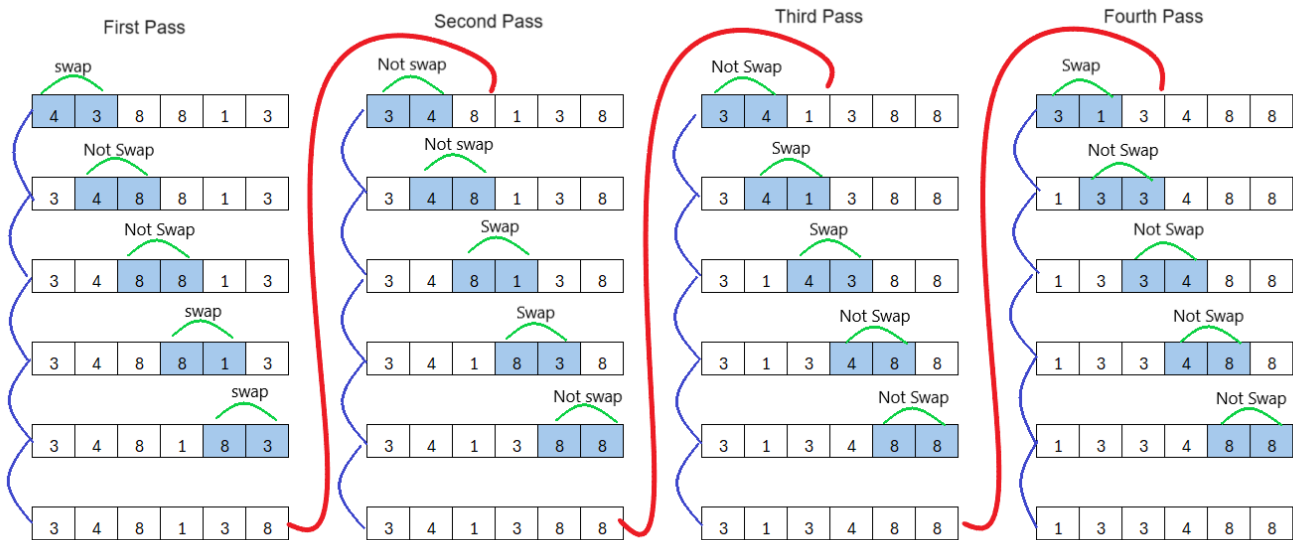
A comparator function does this comparison and the methodology behind it will be explained in the pseudocode and in the accompanying Java programme.

This comparator function takes each pair of numbers and compares the two of them against each other, if the second number is smaller than the first it switches them and then makes a new pair with the second number of the current pair and the next number in the array. The inefficiency that comes with bubble sort is due to the fact that any numbers that have been iterated past cease to exist for comparison reasons and even though the larger and smaller number of each pair is sorted, the smaller number may still have numbers larger than it previously in the array and the only way to access these numbers to compare them is to move through the array again, again comparing pairs of numbers.

Due to the numbers not being fully sorted with each pass this means in order to obtain a fully sorted array, numerous passes must be made until the array is fully sorted. Each pass takes time, and this is where the inefficiency occurs, particularly as the size of the data set enlarges. Bubble sort works very efficiently when using small data sets due to the fact that it requires no additional memory to run the sort, but as the algorithm has to iterate along the whole array numerous times in order to return the sorted list of elements, the efficiency drops as the number of iterations it will have to do in order to fully sort the data will increase. With each iteration the algorithm slowly sorts the data, moving the relevantly largest element (in our case numbers) to the end of the array and the smallest element to the beginning of the array.

The diagram below shows how the algorithm works in a visual form and the pseudo code will explain the methodology behind the Java coding that I have created that accompanies this report.

Bubble Sort Diagram



Bubble Sort Pseudocode

(Reference ATU CTA Notes)

Algorithm 1: Bubblesort

```

1  input: array of numbers or anything else sortable
2  output: The function does not return any value
3  side-effects: the array is modified in-place
4
5  function bubbleSort(A : list of sortable items)
6      n := length(A)
7      repeat
8          swapped := false
9          for i := 1 to n-1 inclusive do
10             // if this pair is out of order
11             if A[i-1] > A[i] then
12                 // swap them and remember something changed
13                 swap(A[i-1], A[i])
14                 swapped := true
15             end if
16         end for
17     until not swapped
18 end function

```

As per both the diagram and the pseudo code Bubble sort works as follows.

- Takes the first 2 elements of the array compares them against each other if element 1 (e1) is larger than element (e2) it swaps them otherwise they stay in the same position.
- It moves on to the next 2 elements e2 and e3 and compares them, if e2 is larger than e3 it swaps them otherwise it moves on to the next 2 elements and e2, e3 remain where they are.
- It continues this swapping until it reaches the end of the array and then it passes(iterates) across the array again repeating the same comparison again.

- The algorithm continues iterating across the array until it reaches an iteration where no swaps occur and then it will return this array.

As I am testing the efficiency of this algorithm, I am more interested in the time that the algorithms took to run based on the data set sized and this is what I have coded to return, rather than the sorted array.

Java code for Bubble Sort

```
public static void bubble(int [] arr) {  
    for (int i = 0; i<arr.length-1; i++){  
        for (int j=0; j<arr.length-i-1; j++){  
            //Compare the adjacent elements  
            if (arr[j]>arr[j+1]){  
                //swap temp and cloned[i]  
                int temp = arr[j];  
                arr[j] = arr[j+1];  
                arr[j+1] = temp;  
                //System.out.println(Arrays.toString(cloned));  
            }  
        }  
    }  
}
```

In this section I will give a description of the algorithm I wrote in order to do a Bubble sort.

The method is declared, and the array used for the method comes from a randomised array that has been declared and computed in a separate function.

Nested inside the method as an inner loop is the comparative function that is used to compare the adjacent elements within the array.

As shown by the for statement the algorithm iterates through the array, one element at a time and by use of an if statement the comparison occurs, checking if the first element of the pair being evaluated is larger than the second element and if it is the swap occurs through the use of the code.

```
if (arr[j]>arr[j+1]){  
    //swap temp and cloned[i]  
    int temp = arr[j];  
    arr[j] = arr[j+1];  
    arr[j+1] = temp;
```

There is a commented out print command at this point to check if the swap has occurred as part of the debugging method as well.

The outer loop of the algorithm is to shorten the range of sorting, as with each iteration the end elements are being sorted and do not need to be checked.

The outer loop runs $n-1$ times, to demonstrate the shortening of the range of elements that need to be sorted. The inner loop compares each element to the element to its right ($j+1$) and as shown by the if statement when $j > j+1$ the above piece of code switches them within the array.

There are $n-1$ iterations through the list, so in the case of the array shown in the diagram, that would be 5 iterations as even though 4 passes are shown, there would be a 5th pass to confirm that all elements are in the correct places and no further swaps are needed and that the algorithm can complete.

As there are $n-1$ passes across the list of n number of items, the total comparisons result in n^2 .

As with all sorting algorithms, the best-case scenario would be if the list appeared already sorted and swaps of elements are needed, but because of the method of this algorithm the comparisons would still occur.

The average case for the Bubble Sort algorithm would be if exchanges were only needed for half the list and if the list was partially sorted.

The worst-case scenario would be if the array was sorted, but in descending order rather than ascending as that would mean that an exchange must take place involving every single pair of elements.

Bubble Sort Analysis

Due to the methodology behind Bubble Sort, no matter how the elements are arranged within the array, there will initially be $n-1$ iterations needed to sort an array of size n .

The table below taken from the book *Problem solving with Algorithms and Data Structures using Java* – Bradley N. Miller, David L. Ranum, Roman Yasinovskyy, J. David Eisenberg displays the number of comparisons per pass.

Pass	Comparisons
1	$n - 1$
2	$n - 2$
3	$n - 3$
...	...
$n - 1$	1

For Bubble Sort the total number of comparisons equals the sum of the first $n-1$ integers.

The Sum of the first n integers is $\frac{1}{2}n^2 + \frac{1}{2}n$

The sum of the first n-1 integers is $\frac{1}{2}n^2 + \frac{1}{2}n - n$ or $\frac{1}{2}n^2 - \frac{1}{2}n$

This is $O(n^2)$ comparisons.

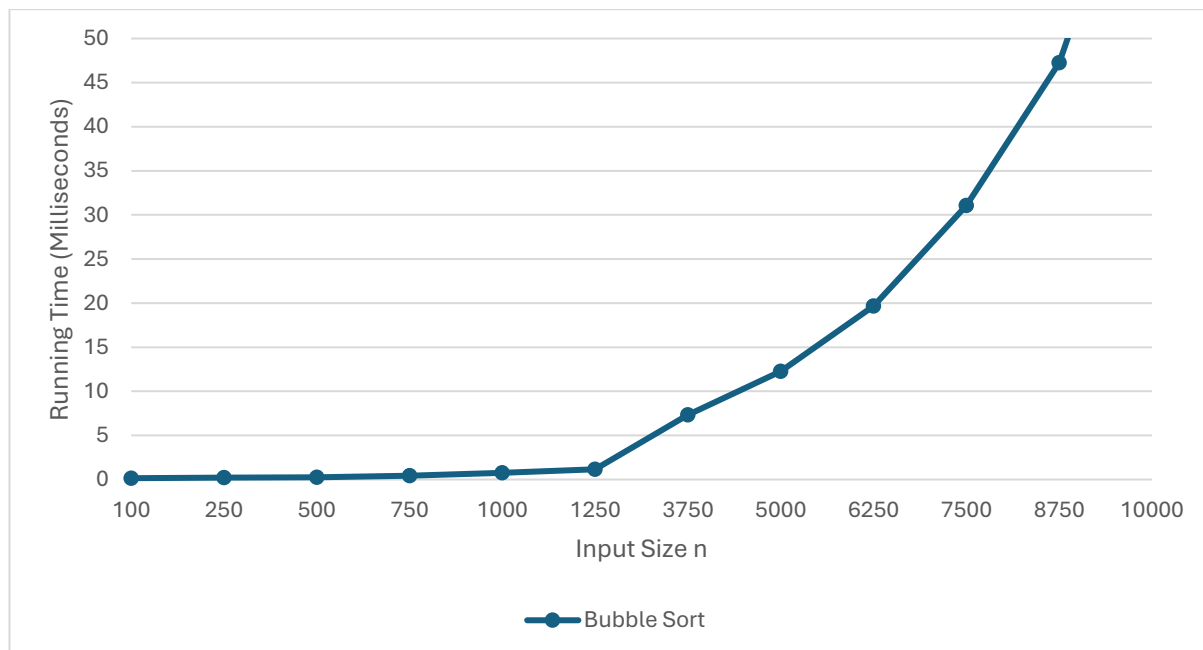
The Bubble sort is classed as one of the most inefficient forms of sorting algorithms because the final location is not known until it has completed. This means that all n-1 iterations must complete regardless of whether the array has sorted or not.

These additional exchange operations are costly, but as the algorithm has to go across the whole array numerous times, this means that unlike some of the other sorting algorithms, it has the capability to end early should all the elements be sorted and there are no further swaps needed.

This modification is known as a short a bubble and a break is inserted when no more exchanges take place. This short bubble sort method, in turn will increase the efficiency of the algorithm.

This inefficiency can be seen clearly in the results I obtained when running my sorting algorithm on arrays of various lengths and the accompanying graph clearly demonstrates the inefficiency as the data set grew in size.

Array Size	0	100	250	500	750	1000	1250	3750	5000	6250	7500	8750	10000
Bubble Sort	0	0.152	0.21	0.244	0.436	0.752	1.161	7.322	12.253	19.66	31.069	47.246	77.52



As previously mentioned, the best-case scenario for bubble sort would be an already sorted array, but even with a sorted array and there needing to be no movement of elements there will be

numerous passes to confirm that all elements are in their final places which means that when it comes to time complexity the best-case complexity would be $O(n)$.

The worst-case scenario as previously mentioned would be an array that appears as sorted in descending order and needed to be sorted in ascending order as every element would need to be moved from its current position in order to be put in its final position.

This means that the number of iterations would be $n-1$ and the number of comparisons would be.

$=(n-1)+(n-2)+(n-3)+\dots+2+1$ as shown above in the table as every number would need to be compared and swapped.

Using the sum of natural numbers formula, $[n(n+1)]/2$, which is what is basically being extrapolated here. This becomes.

$$=(n-1)*(n-1+1)/2$$

$$=(n*(n-1))/2$$

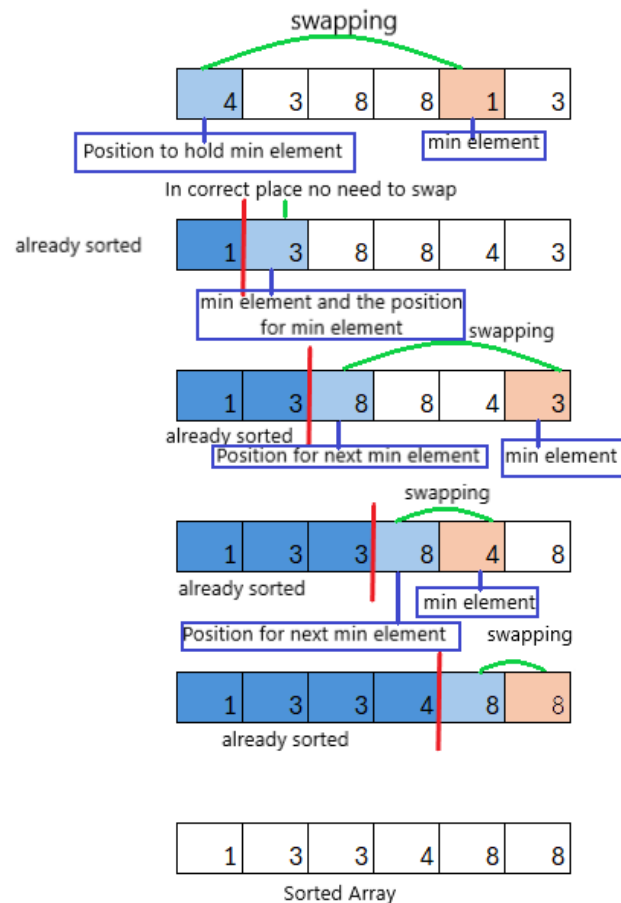
$$=O(n^2)$$

The average case returns the same complexity ($O(n^2)$) due to the fact that the number of comparisons remain the same regardless of the arrangement of elements and is generally applied when the array is presented in a random order.

As bubble sort is an in-place, stable sorting algorithm the space complexity for it is $O(1)$ as additional memory is only required to hold the swapping variable as the swapping occurs.

Selection Sort

Selection sort similar to bubble sort is a simple, in place sorting algorithm. Unlike bubble sort, it doesn't bubble through the list comparing 2 numbers at a time regardless of the positioning of the rest of the array; it instead iterates through the array to find the smallest element and starting with that element it creates a sorted subsection until the whole array has been sorted. It is also not classed as a stable sort because unlike Bubble Sort when two elements are equal in size, they are still swapped using Selection Sorting algorithms, while using Bubble Sort the two elements when equal may be left in the same index rather than swapping them. The diagram below shows how selection sort works in a visual format.



In the diagram above each row of sorting represents a pass across the array. So similar to bubble sort in order to sort the array from start to finish a significant amount of passes are required.

On the initial pass the sort marks the first index of the array as the initial holding place for the minimum number in the array and searches through the array for that minimum number. It does this by working along the array, comparing each number to the number occupying the minimum number slot and using a comparison function, it compares each number against that one until it finds one smaller than it and uses it as the minimum index number. On reaching the end of the first pass of the array this minimum index number is then swapped with the actual number in the minimum number holding place and algorithm begins another pass.

The boundary of the array is moved along so the new minimum number holding index is now one index on and the whole comparison and swapping procedure begins again.

This process is repeated until the array has been fully sorted and the sorted array is returned.

Java Code for the Selection Sorting algorithm

In the algorithm I created in Java as shown below the various parts of the process is highlighted.

```

public static void selection(int[] arr) {
    //moving the boundary of the unsorted subarray
    for (int i = 0; i<arr.length-1; i++){
        int min = i;
        //Find the minimum element in the array
        for (int j = i+1; j<arr.length; j++){
            if (arr[j]<arr[min]){
                min = j;
            }
        }
        //Swap the found minimum element with the first element
        int temp = arr[min];
        arr[min] = arr[i];
        arr[i] = temp;
        // System.out.println(Arrays.toString(cloned));
    }
}

```

The first part of the coded algorithm sets the boundary of the unsorted subarray.

The comparison function is the part shown below where the elements are compared, and the minimum element is switched for the number being held before being placed in the first element of the subarray.

```

//Find the minimum element in the array
for (int j = i+1; j<arr.length; j++){
    if (arr[j]<arr[min]){
        min = j;
    }
}

```

On completion of this inner loop the elements are swapped using the section of algorithm below and the outer loop iterates moving the boundary on again.

```

//Swap the found minimum element with the first element
int temp = arr[min];
arr[min] = arr[i];
arr[i] = temp;
// System.out.println(Arrays.toString(cloned));

```

Selection Sort Analysis

Similar to Bubble Sort the Time Complexity of the Selection Sort is $O(n^2)$ as there are two nested loops and the way the loops function means that outer loop selects an element one at a time from

the array ($O(n)$) and the inner loop compares that element with each other element within the array ($O(n)$).

This means that the total time complexity is $O(n) * O(n) = O(n * n) = O(n^2)$

The Space complexity is similar to the Bubble sort which would be expected as they are both sorts of similar categories, both being in place, simple sorts, however unlike Bubble Sort the selection sort is not a stable sort. Again, the only additional memory is involved in the swapping of the variables within the array making the space complexity $O(1)$ as there is never more than $O(n)$ swaps done.

This would be the case for both average case and worst-case scenarios as they would both require the maximum amounts of iterations and also comparisons.

The best-case scenario would be similar to Bubble Sort in that it would be an already sorted Array that it would be working on, meaning that as the array is already sorted there would be comparisons but no swaps, resulting in a time complexity of $O(n)$ similar to Bubble Sort.

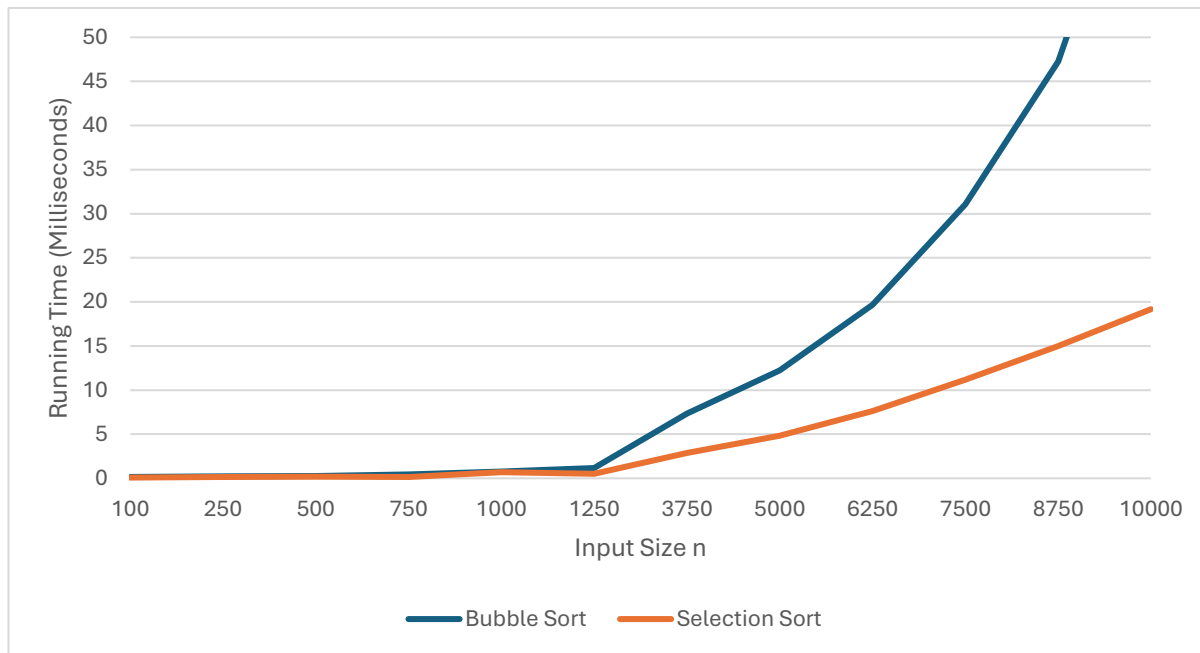
However, unlike Bubble sort because of the way selection sort acts on the data, there would be less exchanges required in both the Worst and average case scenarios, meaning that when it comes to testing for benchmarking purposes selection sorts execute faster typically, which is demonstrated below in both the table and graph based on the data retrieved from the Java programme.

Unlike Bubble sort where the two values when they appear equal, they may be left in their slots and not swapped, Selection Sort will still swap them as if one is larger than the other.

Because of the behaviour of Selection Sort, it too does not scale efficiently as the dataset increases and the larger the data set becomes, the less efficient this sort becomes in time as shown below in the table that was extrapolated from the java code using a benchmarking method of testing various sized data sets, repeatedly for 10 repetitions and an average obtained. Though it is an efficient sort for small data sets, due to the lower demands on overheads and the simple nature of the sort, once the

array size increases the time taken to complete the sorts exponentially increases also.

Array Size	0	100	250	500	750	1000	1250	3750	5000	6250	7500	8750	10000
Bubble Sort	0	0.152	0.21	0.244	0.436	0.752	1.161	7.322	12.253	19.66	31.069	47.246	77.52
Selection Sort	0	0.083	0.159	0.173	0.156	0.71	0.497	2.861	4.842	7.614	11.192	14.985	19.173

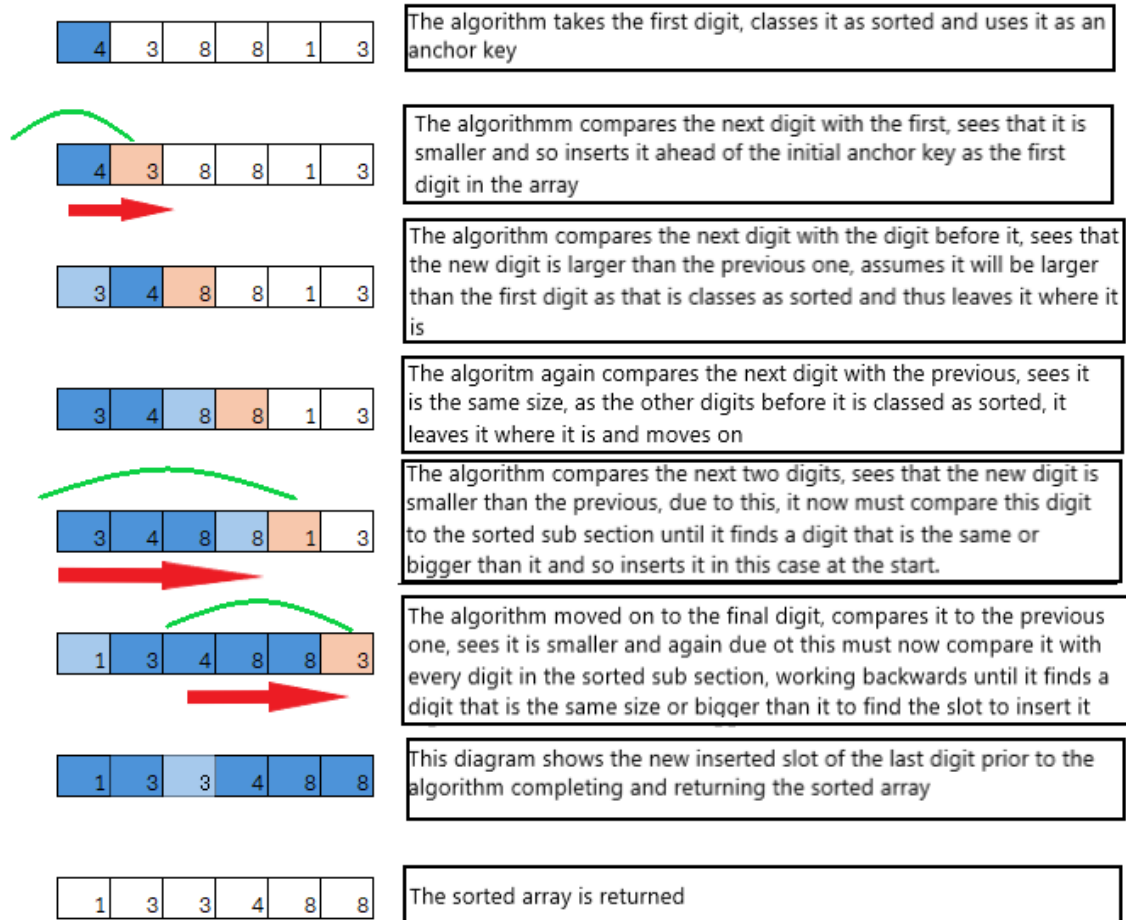


Insertion Sort

Insertion Sort is a simple in-place sorting algorithm. It is a stable sorting algorithm as when it comes across two values of equal amount those two values are not swapped but remain in the appropriate indices as the algorithm sorts by comparing two values and only moving elements if one value is less than the other.

Insertion Sort works very differently to both Selection Sort and Bubble Sort even though they are all classed as in place sorting algorithms.

It works by taking the first element of an array and using it as a point of reference for the rest of the Array and classing that element as sorted. This sorted subarray list grows one element at a time as the algorithm uses a comparative function to iterate through the list and compare each of the elements with each other and inserting them into the sorted sub array in the appropriate slot. As shown below in diagrammatic form.



As any array with a length of one is considered sorted, when the insertion sort assigns the key value to the first index it creates a sorted sub array to begin the sorted list.

The array then begins a second pass, it moves the boundary of the unsorted array along one in order to account for this sorted key index and takes the value next to the key index and compares it with the key index, if this new value is smaller than the key index, it takes the key index, moves it up one slot in the array, creating a space for this new value to slot, or insert into the sorted sub array.

The algorithm then begins its next pass, again moves the boundary of the unsorted array and takes the new first value of the unsorted section and compares it to the last value of the sorted array. If this new value is not smaller, as demonstrated above the new value is left in its current position and the algorithm reiterates back to the start, moves the boundary and continues with the comparing, moving values and inserting the new values in the sorted sub array.

As the algorithm compares the values, unlike Bubble Sort, where only two values are compared or Selection Sort where the first value of the unsorted sub array is used as a holding space and the rest of the unsorted array compared against it; insertion sort takes the value from the unsorted array and compares it with each of the values within the sorted subarray working backwards along the sorted array until it finds a value that it is no longer smaller than and every value it has been compared against is moved along one slot to create a space for the new digit to be inserted into.

The iterating continues through the array, until the whole array is classed as sorted.

Java Programme for Insertion Sort Algorithm

The above described and diagrammatically shown processes can also be seen in the java code that was used in the Benchmarking process which is shown below.

```
public static void insertion(int [] arr) {  
    //moving the boundary of the unsorted subarray  
    for (int i = 1; i<arr.length; i++){  
        //assigning a reference key variable  
        int key = arr[i];  
        int j = i-1;  
        //comparing the elements and creating space for any insertion  
        while ( j>=0 && (arr[j]>key)){  
            arr[j+1] = arr[j];  
            //System.out.println(Arrays.toString(cloned));  
            j--;  
        }  
        arr[j+1] = key;  
    }  
}
```

The first section of the code is moving the boundary of the unsorted array and assigning a key value to reference for the comparison function.

```
    //moving the boundary of the unsorted subarray  
    for (int i = 1; i<arr.length; i++){  
        //assigning a reference key variable  
        int key = arr[i];  
        int j = i-1;
```

The comparator function then compares the elements and moves the rest of the elements that are larger than selected value along to make space for the insertion of any elements of the Array smaller than the any values in the sorted array.

```
        //comparing the elements and creating space for any insertion  
        while ( j>=0 && (arr[j]>key)){  
            arr[j+1] = arr[j];  
            //System.out.println(Arrays.toString(cloned));  
            j--;  
        }
```

The array then reassigns the key value and reiterates back to the start to move the boundary until the whole array has been iterated through and thus sorted.

Insertion Sort Analysis

As the Insertion Sort is an in-place sort that contains an outer and inner loop in order to iterate through the Array and compare each element of the array, the time complexity is similar to the Selection Sort and Bubble Sort.

This is due to again the algorithm having a best case scenario that the Array would be if not completely sorted a vast majority would be in the desired sorted order and the inner loop would not need to iterate at all as it is solely designed to only iterate when it is attempting to create an insertion point and as no insertion point is needed as the elements are already sorted in their final position, thus the inner loop does not iterate.

Of the three in place algorithms discussed today Insertion sort is the only one of the three that has this feature, the other two sorts will still iterate through the inner loop.

For the worst and average case scenarios the time complexity would be deemed to be $O(n^2)$ as it follows a similar pattern to Bubble sort regarding that the number of operations for n values is

$$1+2+3+\dots+(n-1)$$

And using the sum of natural numbers formula that can be written as

$$\frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

As $\frac{n^2}{2}$ term dominates in the above equation, it can be simplified by removing the second term.

And the Big O notation would be returned as follows.

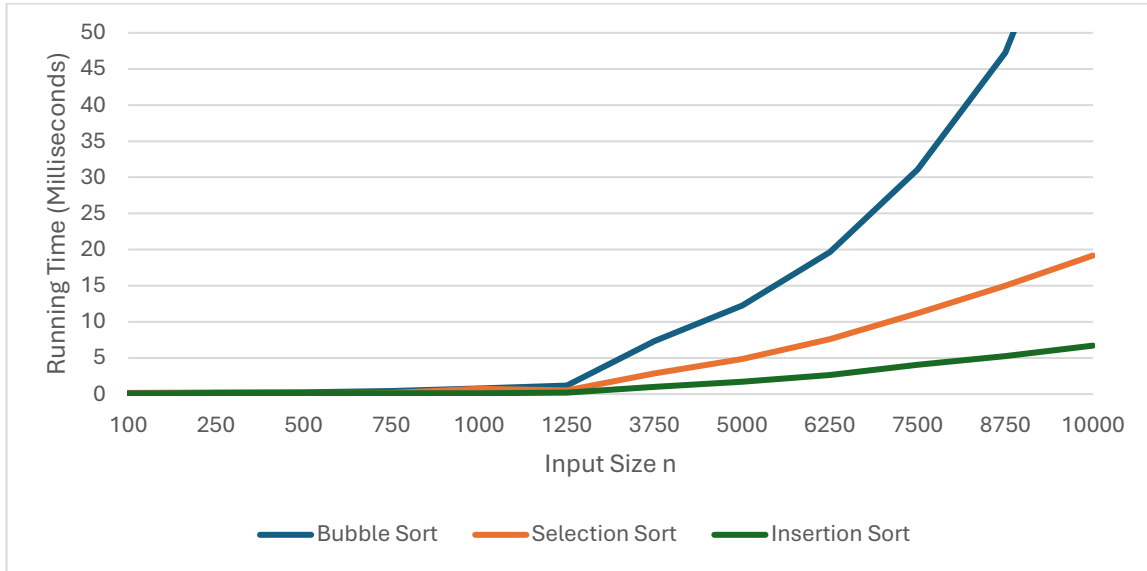
$$O\left(\frac{n^2}{2}\right) = O\left(\frac{1}{2} \cdot n^2\right) = O(n^2)$$

The space complexity for insertion sort is $O(1)$ as the additional key variable means that unlike bubble and selection sort, there is no addition memory required to hold variables during the moving of the values out of the unsorted sub array and into the sorted sub array as no swapping occurs, the values are moved up an index and space is created by the algorithm itself.

This means that of the three in place sorting algorithms insertion sort is the most space efficient.

Like both bubble and selection sort insertion sort is an efficient comparison-based sorting algorithm when using smaller sized data sets or data sets that are almost completely sorted, but as demonstrated by the results obtained from the benchmarking process, once the data set size increases the efficiency drops, though as demonstrated it is still more efficient than both bubble and selection sort.

Array Size	0	100	250	500	750	1000	1250	3750	5000	6250	7500	8750	10000
Bubble Sort	0	0.152	0.21	0.244	0.436	0.752	1.161	7.322	12.253	19.66	31.069	47.246	77.52
Selection Sort	0	0.083	0.159	0.173	0.156	0.71	0.497	2.861	4.842	7.614	11.192	14.985	19.173
Insertion Sort	0	0.048	0.132	0.141	0.113	0.102	0.176	1.011	1.686	2.652	4.062	5.255	6.704



Efficient Comparison Sorting Algorithms

Efficient comparison sorting algorithms are algorithms that compare the elements of the array against each other through various means and return a sorted array, yet the performance does not degrade as the size of the array increases.

An example of an efficient comparison sorting algorithm I will be discussing is the merge sorting algorithm.

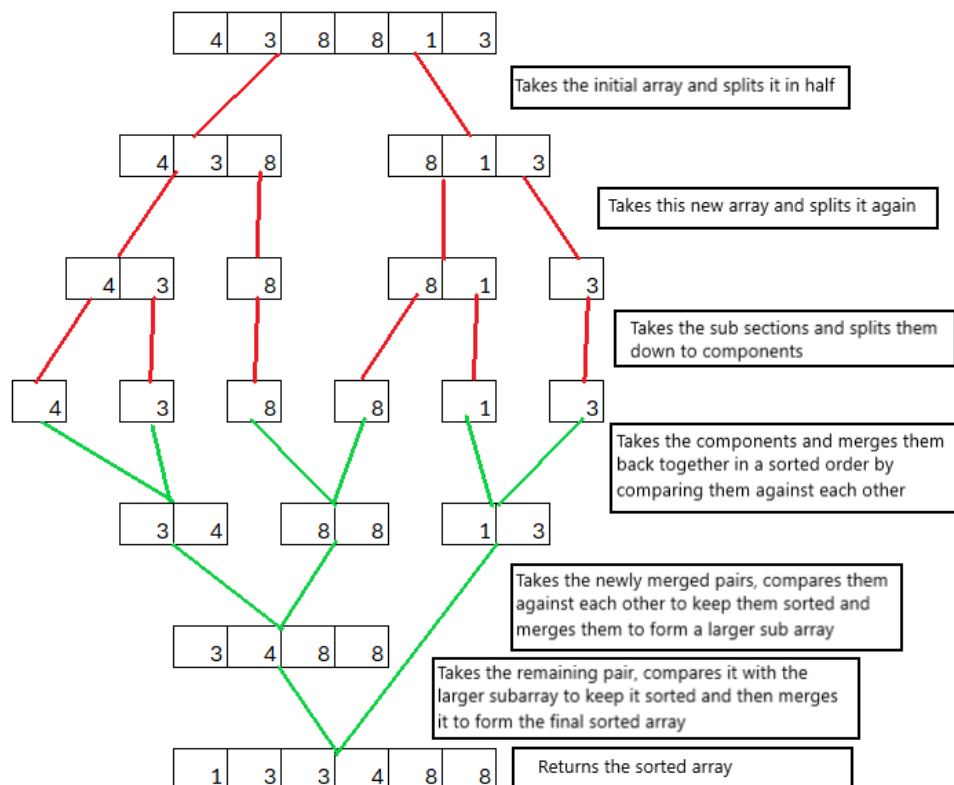
Merge Sort

Merge sort is a sorting algorithm that takes a divide and conquer approach to sorting data sets.

This divide and conquer approach is done by splitting the initial array into half and then recursively dividing those split arrays down until they can be not divided further and under the base case rules, an array of one or less is deemed a sorted array, so through recursively dividing them down it sorts the main array.

These divide subarrays are then merged back together using a second method and when they are merged back together they are compared against each other in order to keep the merged arrays in a sorted order, until all the sub arrays have been merged back into a sorted array of the same length and containing the same elements as the initial unsorted array that was taken in by the first method initially.

The diagram below shows pictorially the way that the merge sort works.



The diagram above shows how the two methods work in the divide and conquer method discussed above.

The first method denoted by the red line acts initially on the full array, splitting it into half.

The method then acts on those two halves by splitting them down again and this continues until the initial array is divided down into its individual components that are by their very nature deemed as individually sorted as an array of one or zero is defined as sorted by merely existing.

The second method then takes those individually sorted sub arrays and by comparing each subarray to each other the method merges them back into sorted pairs. The method then compares the elements in each pair against the elements in the pair beside it and merges them into a larger sorted sub array, before finally taking any remaining subarrays, comparing their individual elements against the elements in the larger subarray and again merging their elements in a sorted output array.

In practical terms, the initial part of this merge sort is done using a recursive function that calls the subarrays to continually be acted upon recursively which is demonstrated in the code that I will past below.

The same method will also call the second method in order to return a sorted array, while providing the inputs for that second method to sort and merge together.

Because of this divide and conquer method and the lack of needing to iterate through an array multiple times in order to sort it, this sorting method is a lot more efficient than the previously mentioned arrays and does not lose its efficiency as the array grows unlike the in-place sorting algorithms previously discussed.

Java Programme for Merge Sort Algorithm

As stated, several times, this algorithm uses two methods in order to sort it. The programming I used for the benchmarking project is shown below and I will explain the various components of it.

This is the initial method that divides the algorithm into its components and also calls the second method to return the sorted array is shown below.

```
public static void mergeSort(int[] arr) {  
    if (arr.length > 1) {  
        //Finding the middle of the array  
        int mid = arr.length / 2;  
        //Creating the left and right subarrays  
        int[] left = Arrays.copyOfRange(arr, 0, mid);  
        int[] right = Arrays.copyOfRange(arr, mid, arr.length);  
        //Sorting the left and right subarrays  
        mergeSort(left);  
        mergeSort(right);  
        //Merging the left and right subarrays  
        merge(arr, left, right);  
    }  
}
```

In the method above, as the arrays when split contained more than one element the function `Arrays.copyOfRange` was used to produce the new subarrays that were needed for the divide and conquer approach to proceed.

These new subarrays were then passed to a second method that is shown below that contained the comparator function and compared the elements of each array and returned the merged sorted version of the initial randomised array.

```
public static void merge(int[] arr, int[] left, int[] right) {  
    int i = 0;  
    int j = 0;  
    int result = 0;  
    //Comparing the elements of the left and right subarrays  
    while (i < left.length && j < right.length) {  
        if (left[i] <= right[j]) {  
            arr[result] = left[i];  
            i++;  
            result++;  
        } else {  
            arr[result] = right[j];  
            j++;  
            result++;  
        }  
    }  
    //Copying the remaining elements of the left subarray  
    while (i < left.length) {  
        arr[result] = left[i];  
        i++;  
        result++;  
    }  
    //Copying the remaining elements of the right subarray  
    while (j < right.length) {  
        arr[result] = right[j];  
        j++;  
        result++;  
    }  
}
```

As shown in the above programme the comparator function shown below compared the elements in each sub array and based on whether they were larger or smaller than each other they were placed in the new subarray in a specific order.

The first part of the comparator function takes the element from the left subarray as long as it is smaller than the element of the right subarray, if this is false then the right subarray is taken.

It is important to note that in the function shown below there is a condition that ensures the algorithm remains stable and that the order of any duplicate elements is maintained.

The condition is.

```
(left[i] <= right[j])
```

By inputting the <= instead of < as the comparator operative ensures the stability and makes sure that the algorithm is fully sorted.

```
//Comparing the elements of the left and right subarrays
while (i < left.length && j < right.length) {
    if (left[i] <= right[j]) {
        arr[result] = left[i];
        i++;
        result++;
    } else {
        arr[result] = right[j];
        j++;
        result++;
    }
}
```

If either of the arrays have no more elements left to compare two other functions are triggered that take the elements that are remaining in either the right or the left array.

If both arrays have no more elements left, the method is completed and passed back to the first method as the first method calls for it.

```
//Copying the remaining elements of the left subarray
while (i < left.length) {
    arr[result] = left[i];
    i++;
    result++;
}

//Copying the remaining elements of the right subarray
while (j < right.length) {
    arr[result] = right[j];
    j++;
    result++;
}
```

Merge Sort Analysis

The analysis of the merge sort is more complicated than the other sorts as there are two distinct methods that each bring their own time and space complexity.

In order to fully analyse the merge sorting algorithm, the complexity must be broken down into the various components.

Initially the splitting of the list is assigned the time complexity value of $\log n$, where n is the length of the array.

The second method has its own time complexity that must be taken into account. As the second method compares each individual component the merge method results in an array of size n that has had n amount of operations performed on it.

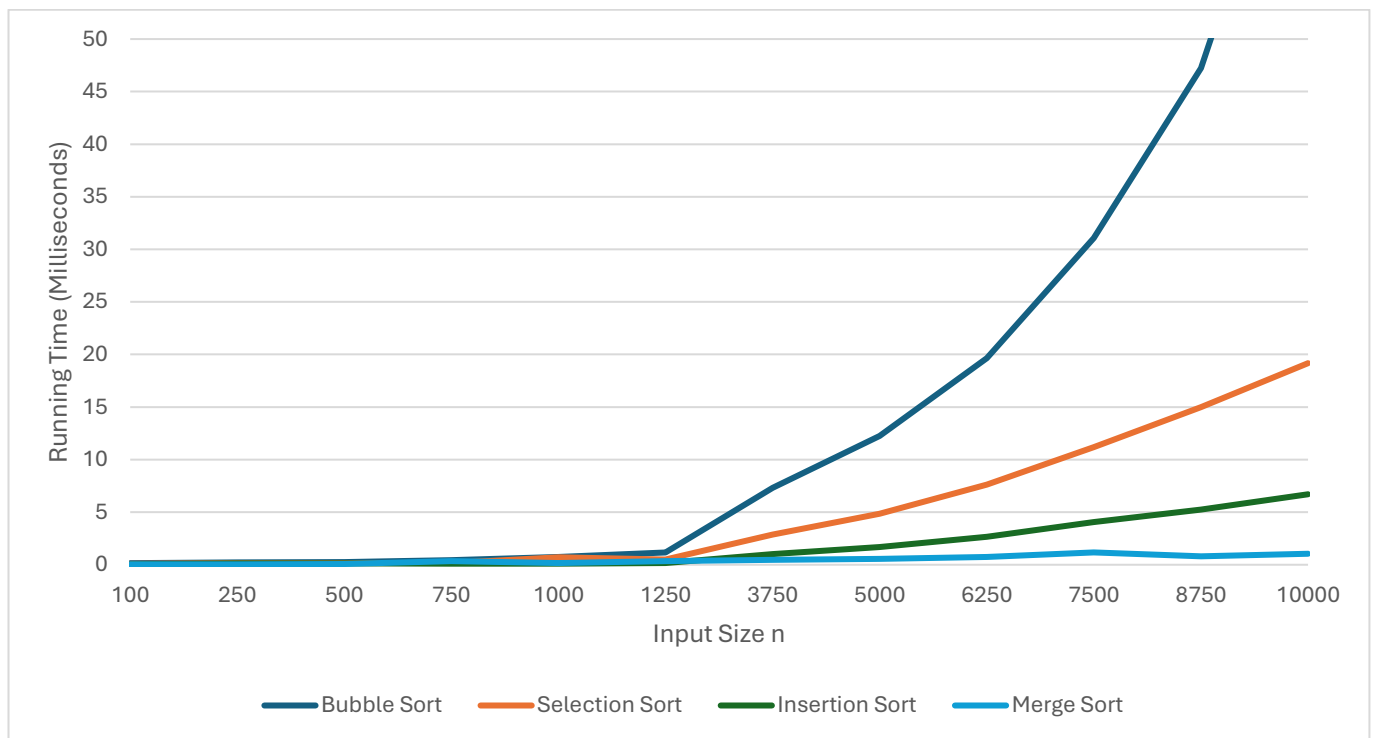
This means that in combining the two distinct sections, $\log n$ splits, each split costs n , for a total of $n \log n$ operations. Giving us a time complexity of $O(n \log n)$.

Space complexity of the merge sorting algorithm is larger than what is required by the in-place sorts. This is due to the of the space needed to store the auxiliary array required during the merging process, which has a size of n . This auxiliary array holds the sorted halves of the initial array, and the initial input is overwritten by this stored auxiliary array.

Due to this additional space requirement this means that the space complexity is equal to $O(n)$.

Even though there is an increase in the space complexity, this sorting algorithm is still a lot more efficient than the in-place sorting algorithms and this efficiency is held even as the data set grows in size as shown in the table and graph below which were produced as a result of the benchmarking process I completed.

Array Size	0	100	250	500	750	1000	1250	3750	5000	6250	7500	8750	10000
Bubble Sort	0	0.152	0.21	0.244	0.436	0.752	1.161	7.322	12.253	19.66	31.069	47.246	77.52
Selection Sort	0	0.083	0.159	0.173	0.156	0.71	0.497	2.861	4.842	7.614	11.192	14.985	19.173
Insertion Sort	0	0.048	0.132	0.141	0.113	0.102	0.176	1.011	1.686	2.652	4.062	5.255	6.704
Merge Sort	0	0.047	0.034	0.064	0.348	0.169	0.338	0.467	0.558	0.729	1.172	0.796	1.049



Non-Comparison Sorting Algorithms

These types of sorting algorithms are completely different to any of the other sorting algorithms that have been discussed through this project.

All the other algorithms have comparator functions that compare the elements of the array in order to sort them.

Non-comparison sorting algorithms simply do not, they work off two simple assumptions according to [Sorting without comparison of elements - GeeksforGeeks](#) (see references for complete web address)

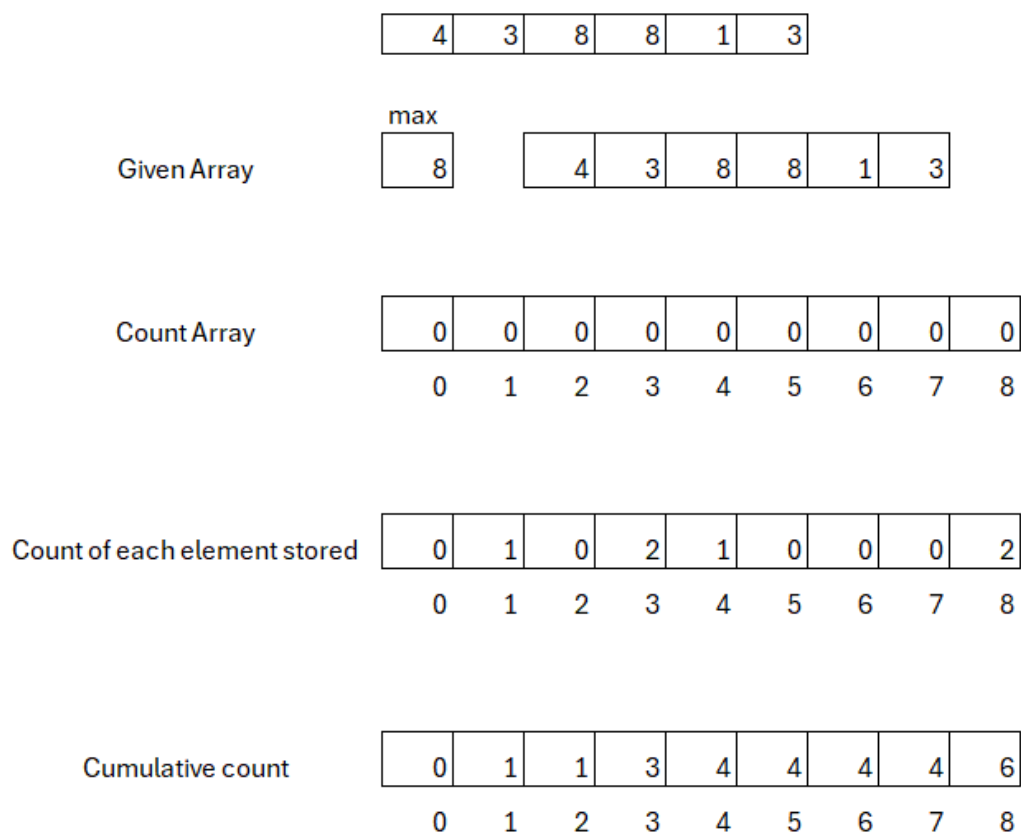
1. All elements of the array are integers.
2. The difference between the maximum and minimum value in the array is less than or equal to 10^6

In this project the example I used for the benchmarking process is the counting sort algorithm, this is a stable non-comparison sorting algorithm.

Counting Sort

The counting sort is a non-comparison based algorithm. It works by counting the frequency of each distinct element within the array and then sorts the various elements into the appropriate index based on this frequency.

It is not a simple algorithm and there are several steps involved in order to sort an array. The best way to explain the steps is with the assistance of the diagrams below.



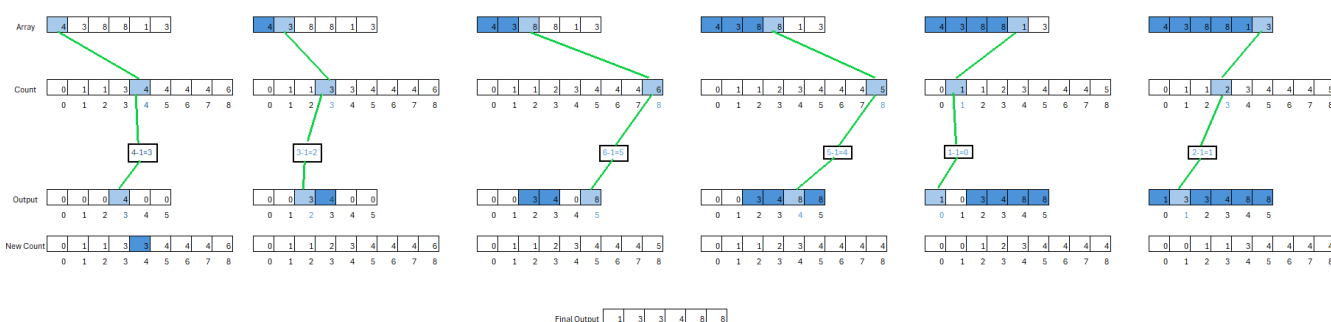
The first steps the counting sort algorithm does is to iterate through the array searching for the largest number. This maximum number is needed in order to initialise the count array and ensure the appropriate amount of slots are held to hold the range of the array.

On finding the maximum value within the array, the algorithm then creates an array the size of the array plus 1 and initialises that array by setting each slot to 0.

The algorithm then iterates through the entire array storing in the count array each time a value appears in the array.

The cumulative count of the various values are then obtained and the sorting begins.

This sorting process I will explain in the following diagram. Please press ctrl+click in order to view a larger version of the diagram.



The algorithm creates and initialises a new array for the output, in this array the algorithm places the appropriate values in the appropriate index based on where it 'should' be once sorted.

The process it uses to do this is by taking the first number of the input array, it then looks at the value that has been placed in the slot for that value, subtracts one from the amount in the count array and that is the index for the new output array for that value. This new amount, (initial amount-1) is then used as the new amount for that value in the counting array and the algorithm iterates on to the next value in the input array.

This iteration continues across the input array until the full array has been completed and the output array is now full of the sorted values from the input array, via the computation done using the count array.

The input array is then overwritten by the new sorted output array and this sorted output array is returned.

This counting sort is completely based on the assumptions initially laid out when discussing the properties of this sort. Even though there are for loops used in the java coding, there is no comparison taking place at all.

Java Programme for Counting Sort Algorithm

For this type of sort there were very clear steps that had to be followed in order for the programme to successfully run.

There is no comparator function, but there are two additional arrays created and the programme itself is more complex than the in-place sorts and I personally found it more logical to write than the merge sort programme.

```

public static void counting(int[] arr) {
    //Finding the largest number in the array
    int max = arr[0];
    for (int i = 0; i<arr.length; i++){
        if (arr[i]>max)
            max = arr[i];
    }
    //initialising the count array with the 0 needed
    int[] count = new int [max+1];
    for (int i = 0; i<max; i++){
        count[i] = 0;
    }
    //Storing the count of each element
    for (int i = 0; i<arr.length; i++){
        count[arr[i]]++;
    }
    //Accumulating the count of the array and storing it
    for (int i = 1; i<=max; i++){
        count[i] += count[i-1];
    }
    //Finding the index of each element of the array and putting
    //them in the output array
    int[] output = new int[arr.length];
    for (int i= arr.length-1; i>=0; i--){
        output[count[arr[i]]-1] = arr[i];
        count[arr[i]]--;
    }
    //copying the output array into the original array
    for (int i = 0; i<arr.length; i++){
        arr[i] = output[i];
    }
}

```

As shown in the picture of the code, I have clearly commented the various parts of the programme, clearly laying out the role of each step and showing how the sorting algorithm works just as shown in the diagrams in programme form.

Counting Sort Analysis

The counting sort algorithm is an efficient algorithm as long as the range of the values (k) is smaller than the number of values (n).

There are very distinct differences between the best and worst-case scenarios and unlike the comparison-based sorts, this efficiency is not based on the size of the array, but rather the size of the range of values for the elements within the array. As the range of values increase the inefficiency of the array also increases and if the range of values are exponentially larger than number of values within the range, this algorithm can become as inefficient as the in-place sorting algorithms when large data sets are introduced to them.

With this in mind in order to calculate the complexity of the algorithm, it is necessary to take into account.

- The fact that the maximum value needs to be found and that requires n operations in order to do so.
- That there must be a counting array created and initialised: As previously stated, the maximum value is k, and this array must be k+1 in size.
- As every value must be not only counted, but also removed, this means the number of operations is two operations per count giving 2*n operations in total.
- The output array must be built and filled with the sorted elements; this gives n operations as n elements must be created.

With these factors in mind the time complexity is calculated as follows

$$\begin{aligned}
 \text{Operations} &= n + (k + 1) + (2 * n) + n \\
 &= 4 * n + k + 1 \\
 &\approx 4 * n + k
 \end{aligned}$$

The simplified expression based on big O notation and the above gives.

$$\begin{aligned}
 O(4 * n + k) &= O(4 * n) + O(k) \\
 &= O(n) + O(k) \\
 &= O(n + k)
 \end{aligned}$$

This would give us the time complexity for counting sort based on the average case scenario.

As previously stated, there is a significant difference between the best, average and worse case scenarios.

The best-case scenario would be that the range (k) would be a fraction of number of values (n)

For example,

$$k(n) = 0.1n$$

This would give a range of 0-10 for 100 values, or a range of 0-100 for 1000 values.

Using the big O notation this would give.

$$O(n + k) = O(n + 0.1n) = O(1.1n)$$

Simplified gives **O(n)**

The worst-case scenario would be if the range(k) is for example the number of values (n) squared.

Example 10 values, but a range of 0-100, or 100 values and a range of 0-10000.

This would give the following computation.

$$\begin{aligned}
 k(n) &= n^2 \\
 O(n + k) &= O(n + n^2)
 \end{aligned}$$

Simplified this give **O(n²)**

The space complexity for the counting sort is also higher than all the previous sorts discussed due to the creation of two arrays during the sorting process. These two arrays would require auxiliary space and as the counting array is of the size of the range(k) and the output array is the size of the input array (n) this would imply that the space complexity of the counting sort is $O(k+n)$.

Overall counting sort is a very efficient sorting algorithm, particularly compared to the comparison-based algorithms as long as the range of the array is much smaller than the number of values within the array. Because it doesn't require comparator functions it is easier to write programmes for and it is a stable algorithm. However, it does have its disadvantages like it doesn't work for any numbers with decimals.

It becomes extremely inefficient if the range becomes larger than the number of values, and this inefficiency grows exponentially as this range grows larger than the number of values.

As a counting sort is not an in-place sorting algorithm it requires additional space for the relevant arrays required during the sorting process, this space can also become rather larger dependent on both the amount of values used or the range and in turn this can cause issues with overhead costs.

Implementation and the Benchmarking process

For this project the main aim was to create an application that analysed 5 different sorting algorithms based on the benchmarking process using Java programming language.

As I have discussed throughout this document and shown various sections of the code used, the 5 sorting algorithms used were.

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Merge Sort
5. Counting Sort

The benchmarking results from the algorithmic code used, were presented as both a graph and a table that was based on the performance of each sorting algorithm using various sized random arrays. Each array was sorted 10 times and an average time taken based on the 10 repetitions.

The main application used is named Sorting.java and the main purpose for the application was to take various sized randomly generated arrays, input them into each of the sorting algorithms, time how long each algorithm took to sort the array and in order to get an average, repeat this process 10 times per array.

I created the application using a package called BenchmarkingProject.src, I imported the java.util library and created a class called Sorting.

Inside this class I created a method to generate the random arrays called randomArray and a method that copied the randomly generated array in order to ensure that for each repetition and for

each of the sorting methods that the method worked on an unsorted random array and not sorted data, called copyArr.

I created a main method which I stored all the data for each of the sorting algorithm methods, including the for loop needed to generate the different sized arrays and the relevant start and end time functions to record the time each sorting algorithm took for the various arrays.

I also added a piece of code that allowed the results to be outputted to a table.

For each of the sorts a method was created appropriately named based on the sorting algorithm being used, except for merge sort, which two methods were created for.

Initially for each sorting algorithm I placed all the code for the repetitions in order to get the average time and the start and end time functions that came from the java.util library. This however became messy particularly when the different size arrays were introduced, and I thought it may be a better practice to store this in the main method alongside the call function for each method.

As shown in the additional code and below, the main menu has segmented call functions for each sorting algorithm, but before the function is called, the startTime parameter, documents the time that each sort began and on completing the sort and returning a sorted array, the endTime parameter again recorded the time.

The startTime was then subtracted by the end time and as the output for both of those parameters were in nanoseconds and in order to give an accurate comparison, milliseconds were required so this value was then divided by 1,000,000.

As I wanted the average time taken, in the section that is used for the table output, this total time was then divided by the number of reps each algorithm was sorted and outputted in the table form.

When testing each of the sort algorithms, print statements were inputted at various points to ensure the programmes were running in the desired way and the outputs at the print statements, were as expected.

At points during the implementation of the code I did find a few problematic areas that took additional time to figure out, for example automating the different sized arrays and not manually changing the value of the array size. The merge sort method caused some consternation as it wasn't sorting the subarrays and returning a merged array exactly the same as the input array, but after some problem solving, I rectified these issues.

Creating the table for the output was another challenging area I found and through trial and error I was able to ensure the table outputted. I did try to figure a way to get the program to output to a spreadsheet, but unfortunately it was not something I could do, so ended up copying and pasting the table results into a text file and then importing the data from that text file to the excel spreadsheet.

Java Benchmarking main function code

Below is the main function of the Java application that I used to put all the various benchmarking code together in.

Run | Debug

```
public static void main(String[] args) {  
    // Array sizes to test  
    int[] arraySizes = {100, 250, 500, 750, 1000, 1250, 3750, 5000, 6250, 7500, 8750, 10000};  
    int reps = 10;  
  
    // Print table header  
    System.out.println("Array Size | Bubble Sort | Selection Sort | Insertion Sort | Merge Sort | Counting Sort");  
  
    // Perform repetitions for each array size  
    for (int size : arraySizes) {  
        double totalBubble = 0;  
        double totalSelection = 0;  
        double totalInsertion = 0;  
        double totalMergeSort = 0;  
        double totalCounting = 0;  
  
        // Perform repetitions  
        for (int i = 0; i < reps; i++) {  
            int[] arr = randomArray(size);  
            long startTime, endTime;  
  
            // Bubble Sort  
            int[] cloned = copyArr(arr);  
            startTime = System.nanoTime();  
            bubble(cloned);  
            endTime = System.nanoTime();  
            totalBubble += (endTime - startTime) / 1000000.0;  
  
            // Selection Sort  
            cloned = copyArr(arr);  
            startTime = System.nanoTime();  
            selection(cloned);  
            endTime = System.nanoTime();  
            totalSelection += (endTime - startTime) / 1000000.0;  
  
            // Insertion Sort  
            cloned = copyArr(arr);  
            startTime = System.nanoTime();  
            insertion(cloned);  
            endTime = System.nanoTime();  
            totalInsertion += (endTime - startTime) / 1000000.0;  
  
            // Merge Sort  
            cloned = copyArr(arr);  
            startTime = System.nanoTime();  
            mergeSort(cloned);  
            endTime = System.nanoTime();  
            totalMergeSort += (endTime - startTime) / 1000000.0;  
  
            // Counting Sort  
            cloned = copyArr(arr);  
            startTime = System.nanoTime();  
            counting(cloned);  
            endTime = System.nanoTime();  
            totalCounting += (endTime - startTime) / 1000000.0;  
        }  
  
        // Print results for each array size  
        System.out.printf(format: "%10d | %11.3f | %14.3f | %15.3f | %9.3f | %12.3f\n", size,  
            totalBubble / reps, totalSelection / reps, totalInsertion / reps,  
            totalMergeSort / reps, totalCounting / reps);  
    }  
}
```


Benchmarking results for the five sorting algorithms

Below are the benchmarking results in tabular form.

Array Size	100	250	500	750	1000	1250	3750	5000	6250	7500	8750	10000
Bubble Sort	0.152	0.21	0.244	0.436	0.752	1.161	7.322	12.253	19.66	31.069	47.246	77.52
Selection Sort	0.083	0.159	0.173	0.156	0.71	0.497	2.861	4.842	7.614	11.192	14.985	19.173
Insertion Sort	0.048	0.132	0.141	0.113	0.102	0.176	1.011	1.686	2.652	4.062	5.255	6.704
Merge Sort	0.047	0.034	0.064	0.348	0.169	0.338	0.467	0.558	0.729	1.172	0.796	1.049
Counting Sort	0.01	0.02	0.035	0.05	0.019	0.058	0.015	0.018	0.025	0.023	0.026	0.036

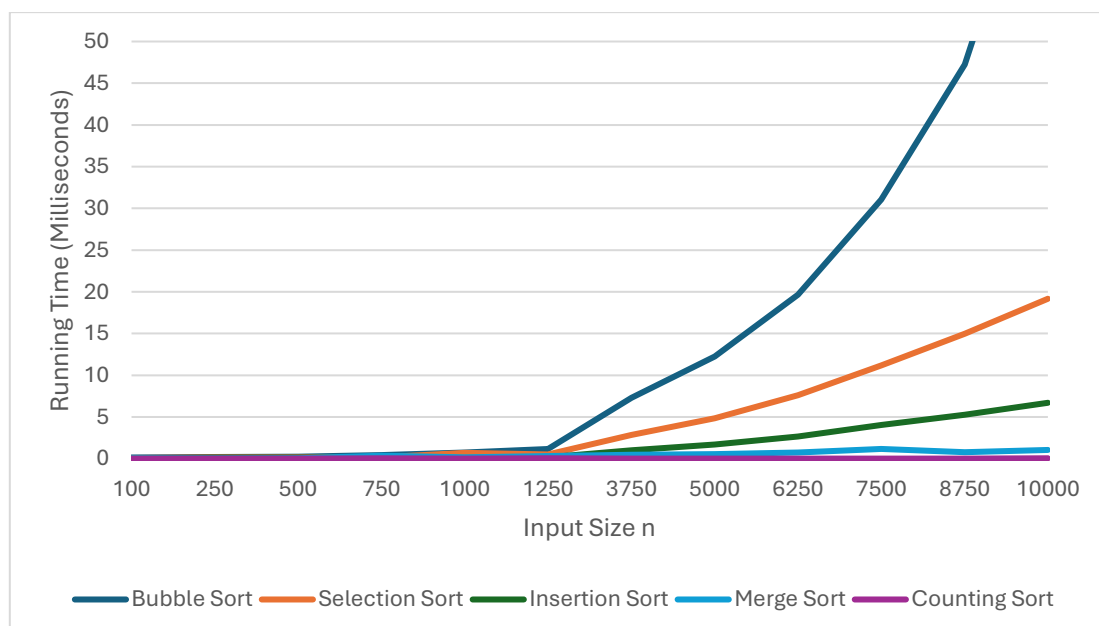
The above table shows the results in milliseconds for the average time taken for each sort based on the various array sizes listed above the top.

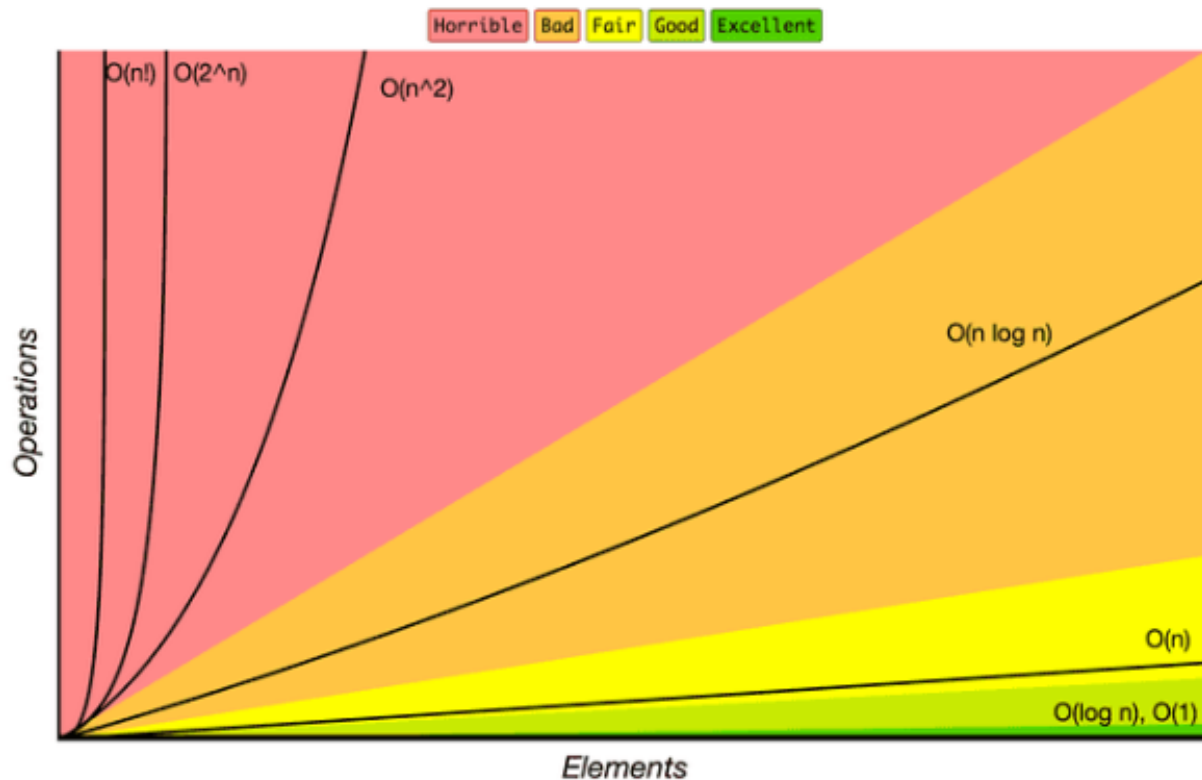
The table does clearly show what has been theoretically stated throughout this project and the sorts did perform in the ways I had expected them to. As shown in the table Bubble sort, selection sort and insertion sort performed fairly similar to the more efficient merge sort and counting sort, but as the array sizes grew their efficiency dropped dramatically.

As shown in the table, as the array sizes grew up to the 10,000 sizes of values, the time taken by the in-place sorts dramatically differed from the merge and counting sorts, with Bubble sort being almost 77 times slower than merge sort and 2000 times slower than counting sort.

This was expected due to the methods the sorts used and also due to the fact that the counting sort didn't have a change in the range of values, only the number of values, hence demonstrated above, that the counting sort remained practically unchanged by the change in the number of elements as the range didn't change alongside the increase in number of elements.

The table was then plotted on a graph, which reinforces the theory behind these sorting algorithms and as shown below are similar in representation as the big O notation would theorise.





One of the noticeable things as shown when comparing the two graphs, is that for the counting sort, the way this benchmarking process worked, the best-case scenario was fulfilled, which I don't believe gives a true portrayal on the behaviour of that sort as it is so heavily dependent on the range of the values, rather than the number of values.

References

Reference list

baeldung (2018). *Merge Sort in Java* | Baeldung. [online] www.baeldung.com. Available at: <https://www.baeldung.com/java-merge-sort>.

callmehk (2018). *Sorting without comparison of elements*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/sorting-without-comparison-of-elements/>.

McKenzie, C. (2022). *How to format a Java table with printf example*. [online] www.theserverside.com. Available at: <https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/Java-print-table-format-printf-chart-console-scanner-println-line>.

Miller, B.N., Ranum, D.L., Yasinovskyy, R. and Eisenberg, J.D. (2023). *DS Problem Solving with Algorithms and Data Structures using Java*. [online] *runestone.academy*. Franklin Beedle & Associates. Available at: <https://runestone.academy/ns/books/published/javads/javads.html> [Accessed 25 May 2024].

Risha (2024). *Learn Data Structures and Algorithms | DSA Tutorial*. [online] GeeksforGeeks. Available at: <https://www.geeksforgeeks.org/learn-data-structures-and-algorithms-dsa-tutorial/?ref=shm>.

www.w3schools.com. (n.d.). *DSA Counting Sort Time Complexity*. [online] Available at: https://www.w3schools.com/dsa/dsa_timecomplexity_countsort.php [Accessed 25 May 2024].